

AUTHORED BY: ARPIT RAJ, LNMIIT JAIPUR

DATE: AUGUST 4, 2026 | © ALL RIGHTS RESERVED

Chapter: LeetCode 695 - Max Area of Island

Overview: Below is the comprehensive, highly organized C++ implementation for solving the *Max Area of Island* problem using a Disjoint Set Union (DSU) data structure. The code is structured for clarity and optimal readability while maintaining exact logic.

1. Disjoint Set Union (DSU) Class

```
class dsu {
public:
    vector<int> parent;
    vector<int> size;

    dsu(int n) {
        parent.resize(n);
        size.resize(n, 1);

        for(int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    int findparent(int node) {
        if(parent[node] == node) {
            return node;
        }
        parent[node] = findparent(parent[node]);
        return parent[node];
    }

    void unionit(int u, int v) {
        int pu = findparent(u);
        int pv = findparent(v);

        if(pu == pv) return;

        if(size[pu] < size[pv]) swap(pu, pv);

        parent[pv] = pu;
        size[pu] += size[pv];
    }
};
```

2. Main Solution Class

```

class Solution {
public:
    int maxAreaOfIsland(vector<vector<int>>& grid) {
        int n = grid.size();
        int m = grid[0].size();

        vector<pair<int,int>> dir = {{-1,0}, {0,-1}, {1,0}, {0,1}};
        dsu ds(n * m);

        for(int r = 0; r < n; r++) {
            for(int c = 0; c < m; c++) {
                if(grid[r][c] == 0) continue;
                int curr = r * m + c;

                for(auto &it : dir) {
                    int nr = r + it.first;
                    int nc = c + it.second;

                    if(nr >= 0 && nc >= 0 && nr < n && nc < m) {
                        int next = nr * m + nc;
                        if(grid[nr][nc] == 1) {
                            ds.unionit(curr, next);
                        }
                    }
                }
            }
        }

        int ans = 0;
        for(int r = 0; r < n; r++) {
            for(int c = 0; c < m; c++) {
                if(grid[r][c] == 1) {
                    int curr = r * m + c;
                    int boss = ds.findparent(curr);
                    int currsize = ds.size[boss];

                    ans = max(ans, currsize);
                }
            }
        }

        return ans;
    }
};

```